

Redis 会话记忆 · 手把手教程

给「agent扫地机器人」项目接入多轮对话记忆 | 预计用时 30 分钟 | 适合：已跑通 FastAPI + Docker 的你

第0步：先搞懂——为什么要会话记忆

你的 Agent 现在是「金鱼记忆」：看 agent/react_agent.py 的 execute_stream，每次只把你当前这句话发给大模型，它不记得你上一句说了什么。

```
# agent/react_agent.py (现在的样子)
def execute_stream(self, query: str):
    input_dict = {
        'messages': [
            {'role': 'user', 'content': query}, # ← 只有当前这一句
        ]
    }
    for chunk in self.agent.stream(input_dict, ...):
        ...
```

多轮对话的原理很简单：把之前的历史消息一起发给大模型。问题是历史存在哪？

- 存在内存里 → 服务器重启就丢，多进程还共享不了
- 存在文件里 → 慢，还要处理并发
- 存在 Redis 里 → 快（纯内存）、自带过期时间、专业做法

Redis 是一个跑在内存里的数据库，专门干这种“临时但高频读写”的活。你的 docker-compose.yml 里已经有 redis 容器了（之前教程预留的），这步就是把它用起来。

第1步：启动 Redis

1.1 启动容器（用你项目里现成的）

你的 docker-compose.yml 里已经有 redis 服务了，直接启动：

```
cd C:\Users\HP\PycharmProjects\PythonProject3\agent扫地机器人
docker compose up -d redis
```

1.2 验证 Redis 活了

```
docker compose ps      # 看到 redis 容器状态是 Up
docker exec -it agent扫地机器人-redis-1 redis-cli ping # 返回 PONG 就成功
```

如果容器名不对，用 docker ps 看真正的容器名，把 redis-cli ping 前面的名字换成它。

第2步：安装 Python 的 redis 库

容器是服务端，你的代码要连它还需要一个客户端库（redis-py）。在项目目录、venv 激活状态下执行：

```
C:\Users\HP\PycharmProjects\PythonProject3\venv\Scripts\activate
pip install redis
```

并把它写进 requirements.txt（在文件末尾加一行）：

```
# requirements.txt 末尾追加
redis>=5.0
```

注意：redis 这个容器服务和 redis 这个 Python 库是两回事，都要有。容器 = 仓库，Python 库 = 你手里的取货凭证。

第3步：写记忆管理模块 `utils/memory.py`

在 `utils/` 下新建 `memory.py`，负责「存消息」和「取历史」两件事：

```
# utils/memory.py
import json
from redis import Redis
from utils.env import settings

# 连接 Redis（URL 在 .env 的 REDIS_URL 里）
r = Redis.from_url(settings.redis_url, decode_responses=True)

MAX_HISTORY = 10 # 最多记住最近 10 轮对话（20 条消息）

def save_message(session_id: str, role: str, content: str) -> None:
    """把一条消息追加到该会话的历史列表尾部"""
    key = f"chat:{session_id}"
    msg = json.dumps({"role": role, "content": content}, ensure_ascii=False)
    r.rpush(key, msg) # 追加到列表尾部
    r.ltrim(key, -MAX_HISTORY * 2, -1) # 只保留最近 N 条，防止无限膨胀

def get_history(session_id: str) -> list:
    """取出该会话的全部历史消息（按时间顺序）"""
    key = f"chat:{session_id}"
    raw = r.lrange(key, 0, -1)
    return [json.loads(m) for m in raw]

def clear_history(session_id: str) -> None:
    """清空该会话历史"""
    r.delete(f"chat:{session_id}")
```

逐行解释：

- `Redis.from_url(...)` — 从 `REDIS_URL` 建立连接；`decode_responses=True` 让返回的是字符串而不是字节，中文不乱码
- `rpush(key, msg)` — 把消息追加到 `key` 对应的列表末尾（`key` 就是 `chat:你的会话ID`）
- `ltrim(key, -20, -1)` — 列表只留最后 20 条，历史太长时自动丢掉最旧的，防止 Redis 被撑爆
- `json.dumps` — 消息先转成 JSON 字符串再存（Redis 只能存字符串）

第4步：把记忆接进对话流程

4.1 改 `agent/react_agent.py` —— 接收历史消息

给 `execute_stream` 加一个 `history` 参数，把历史拼到 `messages` 前面：

```
# agent/react_agent.py（改 execute_stream 一个方法）
def execute_stream(self, query: str, history: list | None = None):
    messages = []
    if history:
        messages.extend(history) # 之前的对话历史
    messages.append({"role": "user", "content": query}) # 当前这句
```

```

input_dict = {'messages': messages}
for chunk in self.agent.stream(input_dict, stream_mode='values', context={'report': False}):
    latest_message = chunk['messages'][-1]
    if latest_message.content:
        yield latest_message.content.strip() + '\n'

```

4.2 改 api/server.py —— 存取历史

在 /chat 接口里：先取历史 → 带着历史回答 → 把问答存回去：

```

# api/server.py (改 chat 函数 + 加一行 import)
from utils import memory # 文件顶部加这行

@app.post('/chat', response_model=ChatResponse)
async def chat(req: ChatRequest):
    start = time.time()
    session_id = req.session_id or str(uuid.uuid4())

    history = memory.get_history(session_id) # 1. 取出这个会话的历史
    result = []
    for chunk in agent.execute_stream(req.query, history=history): # 2. 带着历史回答
        result.append(chunk)
    answer = '\n'.join(result)

    memory.save_message(session_id, 'user', req.query) # 3. 记住用户问的
    memory.save_message(session_id, 'assistant', answer) # 4. 记住你答的
    latency = int((time.time() - start) * 1000)

    return ChatResponse(session_id=session_id, answer=answer, latency_ms=latency)

```

你之前可能没注意：/chat 接口本来就有 session_id 字段——前端传同一个 session_id 就是同一段对话，Redis 就是按它分桶存的。这个设计天生就是为会话记忆准备的。

第5步：验证（3 个测试）

5.1 启动服务

```
uvicorn api.server:app --reload
```

5.2 测试记忆生效

打开 <http://localhost:8000/docs> → /chat → Try it out, 用同一个 session_id 连续提问：

```

第一次：{'query': '我叫小明', 'session_id': 'test-001'}
第二次：{'query': '我叫什么名字?', 'session_id': 'test-001'}
→ 正确回答'小明', 说明它记得

```

5.3 测试会话隔离

```

换个 session_id：{'query': '我叫什么名字?', 'session_id': 'test-002'}
→ 答不上来, 说明两段对话互不干扰

```

5.4 偷看 Redis 里存了什么（加分操作）

```

docker exec -it agent 扫地机器人-redis-1 redis-cli
KEYS chat:* # 看到所有会话的 key
LRANGE chat:test-001 0 -1 # 看 test-001 存的历史消息

```

面试时能说出"我用 redis-cli 验证过数据落库", 瞬间和只会写代码的人拉开差距。

第6步：常见错误排查

附：面试怎么讲这个功能（背下来）

面试官问"你项目里做过什么亮点"，30 秒版本：

"我的 Agent 原来每次对话都是独立的，像金鱼记忆。我用 Redis 给它加了多轮对话记忆：前端每次请求带 session_id，服务端把历史消息存成 chat:{session_id} 的列表，问答时把最近 10 轮历史拼进 prompt 一起发给大模型。Redis 的好处是纯内存读写快，列表结构天然适合消息追加，我还用 ltrim 做了长度控制防止无限膨胀。上线后我用 redis-cli 实际验证过数据落库和会话隔离。"

面试官可能追问，提前准备：

- 为什么用 Redis 不用数据库？会话历史是高频读写、短期数据，Redis 纯内存快 10 倍以上，还能设过期时间自动清理；数据库适合长期结构化数据
- Redis 崩了怎么办？会话记忆丢了可以接受（重开对话），业务数据在数据库里不受影响——这正好体现你分得清"缓存/临时数据"和"持久化数据"
- 为什么只留 10 轮？模型上下文窗口有限，历史太长既慢又贵（按 token 计费），ltrim 控制成本

完成这一步后，你的项目就有：FastAPI 接口 + Docker 部署 + Redis 多轮记忆 —— 面试项目的三块完整拼图。祝面试顺利